

L'Obiettivo del Progetto e l'Approccio Cloud Native

Il progetto consiste nello sviluppo e nel deployment di un "Gestore di Tornei" automatizzato, pensato per orchestrare competizioni sportive o ludiche seguendo le regole del sistema Svizzero, un formato di torneo molto usato in ambiti come scacchi, giochi di carte competitivi (TCG) e sport a squadre, dove non è pratico far giocare tutti contro tutti (girone all'italiana) ma serve comunque un metodo equo per stabilire un classificado finale in un numero limitato di round.

A differenza delle applicazioni tradizionali concepite come software "stand-alone", cioè programmi installati fisicamente su un singolo computer e pensati per essere eseguiti da un solo utente su una sola macchina (si pensi a un vecchio gestionale desktop), questo sistema è stato ingegnerizzato fin dal primo giorno seguendo il paradigma Cloud Native. Questo termine indica un modo di progettare il software pensato appositamente per essere eseguito, gestito e scalato all'interno di infrastrutture cloud, sfruttandone a pieno le caratteristiche elastiche.

In pratica, questo significa che l'applicazione non è un singolo blocco di codice eseguito su una macchina, ma un ecosistema distribuito, cioè composto da più componenti separati che comunicano tra loro attraverso la rete, ospitato sui server remoti di Amazon Web Services (AWS), uno dei principali fornitori mondiali di infrastrutture cloud.

Sfruttando il cloud, il progetto mira a garantire diverse proprietà fondamentali dei sistemi moderni:

- Scalabilità orizzontale: la capacità di aumentare le risorse del sistema non rendendo più potente una singola macchina (scalabilità verticale), ma aggiungendo più macchine o più copie dei servizi in parallelo, per gestire un numero maggiore di utenti contemporaneamente.
- Alta affidabilità (High Availability): la capacità del sistema di rimanere operativo e raggiungibile per la stragrande maggioranza del tempo, minimizzando le interruzioni di servizio.
- Tolleranza ai guasti: il sistema è progettato affinché il guasto di una singola componente (un server fisico, un container, un processo) non comporti l'interruzione totale del servizio. Questo si ottiene distribuendo il carico su più componenti ridondanti, così che se una fallisce, le altre continuano a funzionare mentre quella guasta viene automaticamente ripristinata.

L'obiettivo finale, dunque, non è solo "far funzionare" un gestore di tornei, ma farlo funzionare secondo gli standard delle applicazioni web moderne di livello professionale, dimostrando la comprensione di concetti come distribuzione, resilienza e automazione, che sono al centro delle moderne pratiche DevOps e Cloud Engineering.

Il Passaggio dal Monolite all'Architettura a Microservizi

Storicamente, le applicazioni web venivano sviluppate come un unico enorme blocco di codice, noto come architettura "Monolitica". In un'architettura monolitica, tutte le funzionalità dell'applicazione — la gestione degli utenti, la logica di business, l'accesso al database, l'interfaccia — sono scritte, compilate ed eseguite come un unico programma indivisibile, spesso distribuito come un solo eseguibile o un solo processo.

Questo approccio ha il vantaggio della semplicità iniziale: è più facile sviluppare e testare un singolo progetto. Tuttavia, presenta un limite critico chiamato Single Point of Failure ("punto singolo di rottura"): poiché tutte le funzionalità condividono lo stesso processo e la stessa memoria, un bug anche in una funzione secondaria e poco utilizzata (per esempio, un modulo di reportistica) può generare un'eccezione non gestita che fa crashare l'intero programma, portando offline anche le funzionalità critiche e completamente indipendenti da quel bug.

Oltre a questo, i monoliti sono difficili da scalare in modo selettivo: se una sola funzionalità richiede più risorse (ad esempio un calcolo intensivo), l'unico modo per scalare è duplicare l'intera applicazione, sprestando risorse sulle parti che non ne avrebbero bisogno.

Per ovviare a questi problemi, abbiamo adottato un'Architettura a Microservizi, un approccio in cui l'applicazione viene scomposta in moduli indipendenti, specializzati e distribuiti separatamente, ciascuno responsabile di una singola area di competenza (principio della "Single Responsibility"):

- Frontend: l'interfaccia utente interattiva e reattiva, scritta in HTML, JavaScript e CSS, che viene eseguita direttamente nel browser dell'utente finale ed è responsabile solo della presentazione dei dati e della raccolta degli input.
- Player Service: un microservizio dedicato esclusivamente alla gestione dell'anagrafica dei giocatori (creazione, lettura, aggiornamento, eliminazione dei dati dei partecipanti), sviluppato in Node.js.
- Tournament Service: il motore computazionale del progetto, scritto in Python, responsabile esclusivamente della logica e dell'algoritmo matematico del torneo (creazione dei round, calcolo degli abbinamenti, calcolo della classifica).

In questo ecosistema, i servizi non condividono memoria o processo: comunicano esclusivamente tramite API REST, cioè scambiandosi richieste e risposte HTTP in formato JSON attraverso la rete, esattamente come farebbero client e server separati. Questo disaccoppiamento porta un vantaggio enorme in termini di resilienza: se il Player Service subisce un rallentamento, un bug o un crash completo, il Tournament Service continua a funzionare in modo del tutto indipendente, poiché non condivide risorse con esso. Il guasto rimane isolato al singolo servizio (principio del "fault isolation"), garantendo che il resto del sistema resti operativo anche in condizioni di parziale malfunzionamento — una proprietà semplicemente impossibile da ottenere in un'architettura monolitica classica.

Poliglotta dei Dati (SQL vs NoSQL)

In un'architettura moderna a microservizi non esiste un "database perfetto" universale adatto a ogni scenario applicativo: ogni tipo di dato ha caratteristiche diverse (struttura fissa o variabile, relazioni tra entità, frequenza di scrittura, necessità di coerenza forte o eventuale) e richiede la tecnologia più adatta a quello specifico caso d'uso. Questo approccio, dove ogni microservizio sceglie liberamente il database più adatto alle proprie esigenze invece di forzare un'unica tecnologia su tutto il sistema, è chiamato Polyglot Persistence ("persistenza poliglotta").

Nel nostro progetto abbiamo applicato questo principio dividendo il carico dei dati su due tecnologie profondamente diverse:

PostgreSQL (Relazionale/SQL)

Utilizzato dal Player Service. I dati anagrafici dei giocatori (Nome, Cognome, ID univoco, eventuale Status come "attivo" o "ritirato") hanno una struttura fissa e prevedibile: ogni giocatore ha esattamente gli stessi campi, e questi campi non cambiano nel tempo. I database relazionali come PostgreSQL organizzano i dati in tabelle con colonne rigidamente definite (uno schema), e garantiscono la conformità alle proprietà ACID (Atomicità, Consistenza, Isolamento, Durabilità), un insieme di garanzie che assicurano che ogni operazione sul database sia eseguita in modo affidabile e coerente anche in presenza di accessi concorrenti o guasti. Inoltre, i database relazionali supportano nativamente le relazioni rigide tra tabelle (per esempio un giocatore collegato alle sue partite) e l'integrità referenziale, cioè la garanzia che non possano esistere riferimenti "orfani" a dati inesistenti. Tutte queste caratteristiche li rendono perfetti per dati strutturati e relativamente immutabili come le anagrafiche.

Amazon DynamoDB (NoSQL)

Utilizzato dal Tournament Service. A differenza dell'anagrafica dei giocatori, la struttura di un torneo è altamente dinamica: a ogni round si aggiungono nuove sfide, i punteggi dei giocatori cambiano, si aggiornano i tie-breaker (i criteri di spareggio). Rappresentare questa struttura mutevole con tabelle SQL rigide costringerebbe a frequenti operazioni di modifica dello schema (ALTER TABLE) o a strutture dati complesse e poco efficienti. Un database NoSQL documentale come DynamoDB, invece, offre uno schema flessibile (Schemaless): ogni "documento" salvato nel database può avere una struttura JSON diversa dagli altri, e può essere modificato liberamente senza dover prima aggiornare uno schema condiviso. Questo ci permette di salvare l'intero stato del torneo — round, abbinamenti, punteggi, tie-breaker — come un singolo oggetto JSON coerente, leggendolo e scrivendolo in blocco in modo estremamente rapido, senza la rigidità tipica del mondo relazionale.

La scelta di due tecnologie diverse per due problemi diversi dimostra la comprensione di un principio chiave dell'ingegneria dei sistemi distribuiti: non tutti i dati sono uguali, e la tecnologia di persistenza va scelta in base alla natura del dato, non per abitudine o comodità.

Un ultimo dettaglio tecnico riguarda come le due tecnologie vengono effettivamente interrogate nei nostri microservizi. Nel Player Service, le query verso PostgreSQL vengono eseguite tramite un driver SQL (nel nostro caso "pg" per Node.js), che apre una connessione TCP verso l'istanza del database e invia comandi in linguaggio SQL standard (SELECT, INSERT, UPDATE, DELETE). Il fatto che lo schema sia fisso rende queste query molto efficienti, perché il motore del database può ottimizzare l'accesso ai dati sapendo esattamente come sono organizzati. Nel Tournament Service, invece, le operazioni su DynamoDB non usano un linguaggio di query universale come SQL, ma delle chiamate API specifiche (get_item, put_item, query, scan) esposte dalla libreria boto3 di AWS. DynamoDB, essendo un servizio completamente gestito, si occupa autonomamente di distribuire i dati su più server fisici (sharding) e di garantire tempi di risposta costanti anche al crescere del numero di tornei salvati, un comportamento molto diverso da un database relazionale installato su una singola istanza.

L'Isolamento del Traffico tramite le Porte di Rete

In un ambiente in cui più microservizi girano fisicamente sulle stesse macchine virtuali (o comunque all'interno della stessa infrastruttura di rete), è fondamentale isolare e indirizzare correttamente il traffico affinché ogni richiesta arrivi al servizio corretto e non a un altro per errore. Questo avviene tramite l'assegnazione rigorosa delle Porte Logiche (Ports), un concetto fondamentale delle reti TCP/IP.

Ogni computer collegato a una rete ha un solo indirizzo IP, ma può eseguire contemporaneamente molti programmi diversi che devono comunicare via rete. Per distinguere a quale programma è indirizzato il traffico in arrivo su quell'indirizzo IP, ogni connessione di rete specifica anche un numero di porta, un valore compreso tra 0 e 65535. Immaginando il server (identificato dall'IP) come un grande palazzo, le porte rappresentano i numeri degli appartamenti (i singoli programmi) a cui bussare per essere ricevuti dal servizio giusto.

Nel nostro progetto abbiamo assegnato le porte nel modo seguente:

- Porta 3001: riservata al processo Node.js che espone le API REST del Player Service (creazione, lettura, aggiornamento dei giocatori).
- Porta 5001: riservata all'interprete Python (con il server Flask) che espone le API REST del Tournament Service (creazione dei tornei, generazione dei round, calcolo delle classifiche).
- Porta 5432: la porta standard, riconosciuta a livello internazionale, per la comunicazione con database PostgreSQL. Questa porta viene aperta esclusivamente all'interno dei Security Group configurati su AWS, ovvero i "firewall virtuali" che decidono quali connessioni sono autorizzate a raggiungere l'istanza del database e da quali sorgenti.

Questa netta separazione delle porte permette al Frontend (o a un bilanciatore di carico intermedio) di interrogare esattamente il servizio desiderato, semplicemente indirizzando la richiesta HTTP verso l'IP (o il nome host) corretto seguito dal numero di porta appropriato nell'URL, per esempio "http://indirizzo:3001/players" per i giocatori e "http://indirizzo:5001/tournaments" per i tornei. Senza questa separazione, i tre servizi entrerebbero inevitabilmente in conflitto tra loro, poiché più processi non possono restare in ascolto sulla stessa combinazione di IP e porta contemporaneamente.

Cos'è Docker e il Problema delle Dipendenze

Uno dei problemi più classici e frustranti dell'ingegneria del software è la cosiddetta sindrome del "Sul mio computer funziona" ("works on my machine"). Questo fenomeno si verifica quando un'applicazione funziona correttamente sulla macchina di sviluppo di un programmatore, ma fallisce (o si comporta diversamente) quando viene eseguita su un'altra macchina, per esempio quella di un collega, un server di test, o l'ambiente di produzione finale.

Le cause di questo problema sono molteplici: differenze nella versione del sistema operativo (Windows contro Linux, o versioni diverse dello stesso sistema), differenze nelle versioni delle librerie e dei linguaggi installati (per esempio Node.js 16 contro Node.js 18, che possono avere comportamenti leggermente diversi o funzioni deprecated), variabili d'ambiente configurate diversamente, o semplicemente dipendenze mancanti che sulla macchina originale erano già installate "per abitudine" e non venivano mai documentate esplicitamente.

Docker risolve radicalmente questo problema introducendo il concetto di Containerizzazione. L'idea centrale è che, invece di distribuire solo il codice sorgente dell'applicazione e sperare che l'ambiente di destinazione sia configurato correttamente, si distribuisce un'intera "fotografia" (immagine) dell'ambiente necessario per eseguire quell'applicazione.

Concretamente, Docker "impacchetta" all'interno di un Container:

- Il codice sorgente della nostra applicazione.
- L'ambiente di runtime necessario per eseguirla (per esempio l'interprete Node.js o Python nella versione esatta richiesta).
- Tutte le librerie e le dipendenze software necessarie, già installate e configurate.
- Una versione ridotta e minimale del sistema operativo, sufficiente a far funzionare tutto il resto.

Il risultato è una "scatola magica" completamente autosufficiente, immutabile (una volta creata, non cambia) e isolata dal resto del sistema che la ospita. Grazie a questa proprietà, lo stesso identico container potrà essere eseguito, in modo perfettamente identico e riproducibile, sul portatile personale dello sviluppatore, su un server di test intermedio, o direttamente nei data center di AWS in produzione — eliminando completamente le discrepanze ambientali che affliggono lo sviluppo software tradizionale.

Il Dockerfile, la Ricetta dell'Immutabilità

Per generare un Container, Docker ha bisogno di un insieme di istruzioni chiare, ordinate e riproducibili che descrivano esattamente come costruire quell'ambiente. Queste istruzioni vengono scritte in un file di testo speciale chiamato Dockerfile. Questo documento agisce come una vera e propria "ricetta" automatizzata, dove ogni singola riga definisce un passo (o "layer", strato) nella costruzione progressiva dell'immagine finale.

Analizziamo nel dettaglio un esempio pratico utilizzato per il nostro servizio Node.js (il Player Service):

FROM node:18-alpine

Questa istruzione dichiara l'immagine "base" da cui partire: scegliamo una distribuzione minimale del sistema operativo Linux chiamata Alpine, che pesa pochi megabyte, con Node.js versione 18 già preinstallato al suo interno. Questa scelta è strategica: mantenendo l'immagine sotto i 50 MB circa, si velocizzano notevolmente sia il tempo di costruzione (build) sia il tempo di trasferimento dell'immagine verso i registry Docker e i server cloud.

WORKDIR /app

Con questa istruzione definiamo la cartella di lavoro all'interno del container, cioè la directory in cui verranno copiati i file e in cui verranno eseguiti i comandi successivi. Questo evita di lavorare nella cartella radice del filesystem, mantenendo l'ambiente ordinato e isolato.

COPY package.json index.js ./

Trasferiamo fisicamente i file del nostro progetto (il file delle dipendenze package.json e il codice sorgente index.js) dalla macchina che sta costruendo l'immagine all'interno del filesystem del container, nella cartella di lavoro appena definita.

RUN npm install

Questa istruzione viene eseguita durante la costruzione dell'immagine (non durante l'esecuzione del container) e ordina al gestore di pacchetti npm di scaricare e installare tutte le librerie elencate come dipendenze nel file package.json, rendendole disponibili all'interno del container.

EXPOSE 3001

Con questa direttiva dichiariamo esplicitamente e documentiamo quale porta di rete il processo dentro il container ascolterà per ricevere richieste, in coerenza con il design di rete definito nella sezione dedicata alle porte.

CMD ["node", "index.js"]

Infine, questa istruzione specifica il comando esecutivo principale che verrà lanciato ogni volta che un container basato su questa immagine viene avviato: in questo caso, si esegue il file index.js tramite l'interprete Node.js, avviando il nostro server e mantenendo il container "vivo" per tutta la durata dell'esecuzione del processo.

Il risultato finale di questo Dockerfile è un'immagine Docker completa, pronta per essere costruita una sola volta e poi eseguita in modo identico su qualunque macchina compatibile con Docker.

Il Salto nel Cloud (Amazon Web Services)

Eseguire container Docker in locale, sulla propria macchina di sviluppo, va perfettamente bene durante la fase di scrittura e test del codice. Tuttavia, per offrire un servizio realmente accessibile al pubblico, 24 ore su 24 e 7 giorni su 7, è necessario disporre di hardware sempre acceso, raggiungibile da Internet e adeguatamente mantenuto — un requisito che un normale computer personale non può soddisfare in modo affidabile e continuativo.

Invece di acquistare, configurare e mantenere server fisici propri (un approccio noto come "on-premise", con costi elevati di hardware, energia, spazio fisico e personale tecnico dedicato), abbiamo adottato il modello IaaS (Infrastructure as a Service), noleggiando capacità di calcolo virtuale direttamente da Amazon Web Services (AWS), uno dei principali fornitori mondiali di infrastrutture cloud.

Il Cloud Computing, in generale, ci libera da tutta una serie di responsabilità operative che tradizionalmente gravavano sui team IT: la manutenzione fisica dell'hardware, la sostituzione di componenti guasti, l'aggiornamento del firmware dei server, la gestione della climatizzazione dei data center e persino la gestione della continuità della corrente elettrica (tramite generatori di backup e sistemi ridondanti che il provider cloud gestisce autonomamente).

Nello specifico del nostro progetto, abbiamo scelto di eseguire il deployment dell'infrastruttura nella regione AWS eu-central-1, che corrisponde fisicamente a un data center situato a Francoforte, in Germania. Questa scelta non è casuale, ma motivata da due considerazioni tecniche precise:

1. **Bassa latenza:** poiché il pubblico previsto per l'applicazione è principalmente italiano, ospitare i server in un data center europeo (piuttosto che, per esempio, negli Stati Uniti) riduce significativamente il tempo che i dati impiegano per viaggiare tra il browser dell'utente e i server dell'applicazione, migliorando la reattività percepita.
2. **Conformità normativa:** mantenere i dati fisicamente all'interno del territorio dell'Unione Europea aiuta a rispettare le normative europee sulla protezione dei dati personali (come il GDPR), che impongono vincoli specifici sul luogo di conservazione ed elaborazione dei dati degli utenti europei.

Automazione con Infrastructure as Code (IaC) e Terraform

Creare manualmente reti virtuali, database, cluster Kubernetes e tutte le altre risorse necessarie cliccando attraverso l'interfaccia web (la console) di AWS — un approccio informalmente chiamato ClickOps — è un processo intrinsecamente lento, difficile da documentare con precisione, e altamente soggetto a errori umani. Se un membro del team crea una risorsa con un'impostazione leggermente diversa da quella prevista, o semplicemente si scorda un passaggio, il risultato finale può discostarsi silenziosamente dalla configurazione desiderata, e non esiste una "cronologia" affidabile di cosa sia stato fatto e perché.

Per operare secondo standard professionali propri della disciplina DevOps, abbiamo adottato Terraform, uno strumento di Infrastructure as Code (IaC), letteralmente "infrastruttura definita come codice". Il principio alla base dell'IaC è semplice ma potente: invece di configurare manualmente le risorse cloud tramite interfacce grafiche, si scrive un file di testo che descrive dichiarativamente quale infrastruttura si desidera ottenere, e si lascia che uno strumento automatico (Terraform, in questo caso) si occupi di crearla, modificarla o distruggerla in modo coerente con quanto scritto.

Con Terraform, quindi, descriviamo l'intera infrastruttura di cui abbiamo bisogno — reti, database, cluster — direttamente in file di testo scritti in un linguaggio dedicato. Questo porta un beneficio enorme: l'infrastruttura stessa diventa versionabile su un sistema di controllo versione come Git, esattamente come il codice applicativo. È possibile vedere la storia completa di ogni modifica infrastrutturale, chi l'ha effettuata e perché, e tornare indietro a una versione precedente se qualcosa va storto.

Inoltre, questo approccio garantisce un'enorme resilienza operativa: se il nostro server o l'intera infrastruttura venissero cancellati per errore, oppure colpiti da un evento catastrofico (uno scenario che nella disciplina IT viene chiamato Disaster Recovery, "recupero da disastro"), non è necessario ricordare a memoria — o peggio, ricostruire per tentativi — come fosse configurato l'ambiente originale. Basta rilanciare il comando "terraform apply", puntando ai medesimi file di configurazione, e l'intera infrastruttura verrà ricreata su AWS in modo automatico ed esattamente identica all'originale, nel giro di pochi minuti.

Terraform in Azione (Sintassi HCL)

Per comprendere concretamente la potenza di Terraform, analizziamo il blocco di codice effettivamente utilizzato nel nostro progetto per richiedere il provisioning del database PostgreSQL nel cloud:

```
resource "aws_db_instance" "postgres" {
  identifier = "player-db-cloud"
  allocated_storage = 20
  engine = "postgres"
  instance_class = "db.t3.micro"
  username = var.db_username
  password = var.db_password
  publicly_accessible = true
}
```

Questo codice è scritto in HCL (HashiCorp Configuration Language), il linguaggio di configurazione dichiarativo proprio di Terraform (sviluppato dall'azienda HashiCorp). La parola "dichiarativo" è fondamentale: non stiamo scrivendo una sequenza di comandi imperativi ("crea questo, poi fai quello"), ma stiamo semplicemente descrivendo lo "Stato Desiderato" finale che vogliamo ottenere. È compito di Terraform calcolare autonomamente quali azioni intraprendere per portare l'infrastruttura reale a corrispondere a questa descrizione.

Analizzando i singoli parametri:

- `identifier`: il nome univoco con cui l'istanza del database sarà identificata all'interno di AWS.
- `allocated_storage`: la quantità di spazio disco, in Gigabyte, allocata per il database (in questo caso 20 GB).
- `engine`: il motore di database da utilizzare, in questo caso PostgreSQL.
- `instance_class`: la classe di potenza computazionale della macchina virtuale che ospiterà il database. La scelta di "db.t3.micro" è strategica: si tratta di una delle classi più piccole disponibili, che rientra tipicamente nel piano gratuito (Free Tier) offerto da AWS ai nuovi account, pur fornendo comunque CPU e memoria sufficienti per il carico limitato di un progetto didattico.
- `username` e `password`: le credenziali di accesso al database. È importante notare che, nel nostro codice, questi valori non sono scritti direttamente come testo in chiaro, ma vengono referenziati tramite variabili (`var.db_username` e `var.db_password`). Questo è un accorgimento di sicurezza fondamentale: le variabili vengono fornite a Terraform separatamente (ad esempio tramite un file non versionato o un secret manager), evitando che credenziali sensibili finiscano salvate permanentemente e in chiaro nella storia del repository Git, dove chiunque avesse accesso al codice potrebbe leggerle.
- `publicly_accessible`: un flag booleano che determina se il database sarà raggiungibile direttamente da Internet o solo dall'interno della rete privata virtuale (VPC).

Quando eseguiamo il comando "terraform apply", Terraform legge questa dichiarazione, la confronta con lo stato attuale dell'infrastruttura (che tiene traccia in un apposito "state file"), e contatta autonomamente le API di Amazon Web Services per creare (o modificare) esattamente l'istanza di database descritta, senza che il team debba mai interagire manualmente con la console grafica di AWS.

L'Orchestrazione con Kubernetes (Amazon EKS)

Se Terraform si occupa del cosiddetto Provisioning, cioè di procurare e configurare le risorse fisiche e infrastrutturali di base (i server virtuali, le reti, i database), esiste un problema successivo altrettanto importante: chi si occupa di far girare, distribuire e mantenere in salute il software (i nostri container) sopra a quelle risorse? La risposta è Kubernetes.

Gestire manualmente decine (o centinaia) di container distribuiti su più server, assicurandosi che siano sempre in esecuzione, che vengano riavviati in caso di crash, che vengano aggiornati senza interrompere il servizio, e che il traffico venga distribuito equamente tra le copie disponibili, è un compito semplicemente impossibile da svolgere manualmente su scala reale. Serve un sistema automatico dedicato: un Orchestratore di Container.

Kubernetes agisce esattamente come un "Direttore d'Orchestra" per il nostro sistema distribuito. È un sistema di orchestrazione intelligente e continuamente attivo, che monitora costantemente lo stato di salute di tutti i nostri microservizi, un comportamento noto come Self-Healing ("auto-guarigione"). Concretamente, questo significa che se il container che esegue il Tournament Service dovesse andare in crash — per esempio a causa di un picco improvviso di utilizzo di memoria, un errore conosciuto come Out Of Memory (OOM) — Kubernetes se ne accorgerebbe nel giro di pochi millisecondi, grazie a controlli di salute periodici (health checks), e avvierebbe automaticamente un nuovo container sostitutivo su un nodo sano del cluster, spesso così rapidamente che gli utenti finali non arrivano nemmeno a percepire un'interruzione del servizio.

Per il nostro progetto, non abbiamo installato e gestito manualmente un cluster Kubernetes da zero (un'operazione complessa che richiederebbe di configurare a mano il "Control Plane", cioè il cervello del cluster), ma abbiamo utilizzato Amazon EKS (Elastic Kubernetes Service), la versione completamente gestita di Kubernetes offerta direttamente da AWS. Con EKS, Amazon si occupa autonomamente di mantenere in salute, aggiornare e rendere altamente disponibile il Control Plane del cluster, lasciando al nostro team solo la responsabilità di definire e gestire i carichi di lavoro (i nostri microservizi) che vi vengono eseguiti sopra.

I Fondamenti di Kubernetes: Pod e Deployment

All'interno di Kubernetes, esiste una regola fondamentale da comprendere: un container Docker non viene mai eseguito "nudo e crudo" direttamente dal cluster. Ogni container viene sempre incapsulato all'interno di un oggetto chiamato Pod, che rappresenta l'unità atomica, indivisibile e più piccola gestibile da Kubernetes. Un Pod può contenere anche più container strettamente collegati tra loro (che condividono rete e storage), ma nel nostro caso, trattandosi di un'architettura a microservizi con componenti indipendenti, ogni Pod contiene un solo container.

Un aspetto cruciale da comprendere è che i Pod sono effimeri: Kubernetes può distruggerli e ricrearli liberamente in qualunque momento, per esempio durante un aggiornamento del codice, in seguito a un crash, oppure per ribilanciare il carico tra i nodi del cluster. Proprio per questa natura instabile e temporanea, i Pod non vengono praticamente mai creati manualmente e direttamente da un operatore umano. Si utilizza invece un oggetto di livello superiore chiamato Deployment, che ha la responsabilità di dichiarare e mantenere nel tempo lo stato desiderato di un certo insieme di Pod.

Nel nostro file di configurazione YAML, definiamo un Deployment dichiarando, tra le altre cose, il campo "replicas: 1", che significa letteralmente: "voglio che esista sempre, in ogni momento, almeno una copia esattamente identica in esecuzione di questo particolare servizio". Insieme a questo, specifichiamo anche quale immagine Docker (creata e caricata secondo il processo descritto nelle pagine precedenti) debba essere scaricata ed eseguita all'interno dei Pod gestiti da quel Deployment.

Il comportamento che ne deriva è potentissimo e completamente automatico: se, per qualunque motivo, cancellassimo manualmente uno dei Pod gestiti da questo Deployment (per esempio con il comando "kubectl delete pod"), Kubernetes rileverebbe immediatamente una discrepanza tra lo stato reale del cluster (0 Pod in esecuzione per quel servizio) e lo stato desiderato dichiarato nel Deployment (1 replica). Per risolvere questa discrepanza, il Deployment genererebbe all'istante un nuovo Pod identico, riportando il sistema esattamente allo stato desiderato — senza alcun intervento umano.

Connettività Inter-Cluster: Service e Load Balancer

Come descritto nella pagina precedente, Kubernetes distrugge e ricrea liberamente i Pod per gestire aggiornamenti, guasti e ribilanciamento del carico. Una conseguenza diretta e problematica di questo comportamento è che ogni volta che un Pod viene ricreato, gli viene assegnato un nuovo indirizzo IP privato interno al cluster, diverso da quello precedente. Gli indirizzi IP dei Pod, quindi, cambiano costantemente e in modo imprevedibile.

Questo rappresenta un problema serio: sarebbe assolutamente impraticabile per il Frontend (o per un qualunque altro servizio) dover comunicare direttamente con un backend il cui indirizzo di rete potrebbe cambiare da un momento all'altro, magari anche ogni pochi minuti.

La soluzione architetturale a questo problema è un oggetto Kubernetes chiamato Service. Un Service funge da vera e propria "rubrica telefonica" fissa e stabile, fornendo un nome DNS interno costante (per esempio "player-service") che rimane invariato indipendentemente da quanti Pod nascano o muoiano dietro le quinte. Il Service si occupa autonomamente di mantenere aggiornata, in tempo reale, la lista dei Pod attualmente sani e disponibili a cui indirizzare il traffico, e di distribuire (bilanciare) le richieste in arrivo tra di essi.

Oltre alla necessità di comunicazione interna tra microservizi all'interno del cluster, avevamo anche la necessità opposta: esporre il sito web al pubblico, rendendolo raggiungibile da Internet da qualunque utente esterno al cluster. Per questo specifico scopo, abbiamo configurato un Service di un tipo particolare, chiamato LoadBalancer.

Questa direttiva speciale, quando applicata su AWS tramite EKS, ordina automaticamente ad Amazon di creare un vero e proprio bilanciatore di traffico hardware/software (un Elastic Load Balancer, ELB) posizionato all'ingresso della nostra rete virtuale. Questo bilanciatore ci fornisce un URL pubblico stabile (tipicamente nella forma "xxxx.elb.amazonaws.com"), che gli utenti reali possono digitare nel proprio browser. Il Load Balancer riceve tutto il traffico esterno in arrivo e lo smista, in modo sicuro ed efficiente, verso i Pod del Frontend, che rimangono comunque isolati e protetti all'interno della rete privata del cluster, senza esporre direttamente i loro indirizzi IP interni al mondo esterno.

Sfide Reali nel Cloud: Risoluzione del FailedScheduling

Lavorare con un ambiente cloud reale, e non solo con esempi teorici da manuale, ci ha inevitabilmente posto di fronte a limitazioni architetturali concrete che spesso non vengono discusse nella documentazione introduttiva. Uno degli errori più significativi che abbiamo dovuto diagnosticare e risolvere è stato il messaggio "FailedScheduling - Too many pods".

La causa di questo problema risiede nell'hardware sottostante: i nodi worker AWS che avevamo scelto (di tipo t3.medium, o t3.micro in configurazioni più economiche) hanno limiti hardware rigorosi legati al numero massimo di Interfacce di Rete Elastiche (ENI, Elastic Network Interfaces) che possono essere collegate a ciascuna istanza. Poiché ogni Pod Kubernetes su AWS richiede tipicamente un indirizzo IP proprio fornito tramite queste interfacce, questo si traduce in un limite massimo rigido sul numero di Pod che un singolo nodo può ospitare simultaneamente — un valore che, per istanze di piccola taglia, può essere anche solo di 4 Pod, e che sale a circa 17 per istanze di taglia media come le t3.medium.

Il problema emergeva in modo particolarmente insidioso durante gli aggiornamenti del codice applicativo. La strategia di aggiornamento predefinita di Kubernetes, chiamata Rolling Update, è progettata per garantire zero-downtime (nessuna interruzione del servizio) durante un aggiornamento: per raggiungere questo obiettivo, Kubernetes tenta di avviare i nuovi Pod contenenti la versione aggiornata del codice prima di spegnere quelli vecchi con la versione precedente, così che ci sia sempre almeno una versione funzionante disponibile a servire il traffico.

Il problema è che questa strategia richiede, anche solo temporaneamente, uno "slot" di capacità aggiuntivo rispetto al normale: per un breve periodo, sia i Pod vecchi che quelli nuovi coesistono contemporaneamente. Se il cluster è già vicino al limite massimo di Pod ospitabili sui nodi disponibili (come nel nostro caso, con nodi di piccola taglia), questo surplus temporaneo di richiesta supera il tetto massimo di risorse della macchina, e il nuovo Pod rimane bloccato in uno stato di attesa infinita chiamato Pending, incapace di essere effettivamente pianificato (scheduled) su nessun nodo.

Abbiamo dimostrato capacità concreta di problem solving infrastrutturale risolvendo questo problema in due modi complementari: modificando i manifesti YAML dei Deployment per applicare la strategia alternativa Recreate (che, letteralmente, spegne completamente il Pod vecchio prima di crearne uno nuovo, evitando qualsiasi sovrapposizione temporanea) e, quando necessario, riducendo temporaneamente il numero di repliche di altri servizi non critici per liberare capacità. Questa scelta comporta un sacrificio consapevole: qualche secondo di indisponibilità del servizio durante l'aggiornamento, in cambio della garanzia che il deployment vada effettivamente a buon fine sulle nostre macchine a basso costo, invece di rimanere bloccato indefinitamente.

L'Automazione del Rilascio: CI/CD

Nel moderno ciclo di sviluppo del software professionale, il rilascio manuale del codice — cioè un operatore umano che, da riga di comando sul proprio computer, esegue manualmente i passaggi necessari per costruire e distribuire una nuova versione dell'applicazione — è generalmente considerato un "anti-pattern", ovvero una pratica da evitare. Questo perché i processi manuali favoriscono inevitabilmente la dimenticanza di passaggi, l'inconsistenza tra un rilascio e l'altro, e l'errore umano, specialmente quando il processo viene ripetuto decine o centinaia di volte nel corso della vita di un progetto.

Per ovviare a questo problema, abbiamo implementato una pipeline automatizzata di Continuous Integration e Continuous Deployment, comunemente abbreviata in CI/CD. Questi due termini descrivono due fasi complementari dell'automazione:

- Continuous Integration (Integrazione Continua): il processo automatico di verificare che ogni nuova modifica al codice si integri correttamente con il resto del progetto, tipicamente eseguendo automaticamente test e controlli di qualità ogni volta che il codice viene modificato.
- Continuous Deployment (Distribuzione Continua): il processo automatico, successivo all'integrazione, di distribuire effettivamente la nuova versione validata del codice nell'ambiente di produzione, senza intervento manuale.

Attraverso lo strumento GitHub Actions (un servizio di automazione integrato direttamente nella piattaforma GitHub, dove è ospitato il nostro codice sorgente), abbiamo creato un flusso di lavoro (workflow) completamente automatizzato: ogni volta che un membro del team esegue un'operazione di push (cioè carica) una modifica sul ramo principale (chiamato convenzionalmente "main") della repository, un server temporaneo indipendente, messo a disposizione gratuitamente da GitHub, si attiva automaticamente. Questo server scarica l'ultima versione del codice, ne verifica l'integrità e la correttezza, e avvia in totale autonomia le procedure necessarie per aggiornare l'infrastruttura cloud con la nuova versione dell'applicazione. Il risultato pratico di questa automazione è una drastica riduzione del tempo necessario per portare una modifica dal codice sorgente fino alla produzione: da un processo manuale che potrebbe richiedere decine di minuti (e diversi potenziali punti di errore umano), si passa a un processo automatico che richiede solo pochi secondi di intervento umano iniziale (il semplice comando "git push") e si completa autonomamente in background.

Dissezione della Pipeline GitHub Actions

La nostra pipeline di CI/CD rappresenta un concentrato di automazione sequenziale rigorosamente ordinata. Nello specifico, il file di configurazione YAML del workflow, che risiede nella cartella `./.github/workflows` della nostra repository, definisce ed esegue in sequenza i seguenti passaggi principali (chiamati "Jobs" o "Steps" nella terminologia di GitHub Actions):

1. Checkout

Il primo passaggio consiste nello scaricare (fare il "checkout" di) l'ultima versione del codice sorgente presente sulla repository, direttamente all'interno dell'ambiente temporaneo ed effimero fornito da GitHub per eseguire la pipeline. Senza questo passaggio, il server di automazione non avrebbe alcun accesso al codice da elaborare.

2. Autenticazione Cloud

Successivamente, la pipeline deve dimostrare ad AWS di avere l'autorizzazione per modificarne l'infrastruttura. Questo avviene utilizzando credenziali AWS che, in un progetto ben configurato, non vengono mai scritte direttamente nel codice della pipeline, ma salvate in modo crittografato all'interno di un'area sicura di GitHub chiamata "Secrets". La pipeline recupera queste credenziali in modo sicuro e le utilizza per ottenere un accesso amministrativo temporaneo e limitato al cluster e alle risorse cloud necessarie.

3. Build & Push

Per ciascuno dei nostri microservizi (Frontend, Player Service, Tournament Service), la pipeline ordina a Docker di compilare (build) una nuova immagine Docker, a partire dal Dockerfile aggiornato presente nel codice appena scaricato. Una volta completata la costruzione, l'immagine risultante viene caricata (push) su un registro di immagini pubblico, nel nostro caso Docker Hub, da dove potrà successivamente essere scaricata dal cluster Kubernetes.

4. Deployment (kubect!)

Il passaggio finale consiste nell'invio, tramite il comando `kubect! apply`, dei manifesti Kubernetes aggiornati al Control Plane di Amazon EKS. Kubernetes riceve queste nuove istruzioni, le confronta con lo stato attualmente in esecuzione nel cluster, e — se rileva differenze, per esempio un nuovo tag di versione dell'immagine Docker — innesca autonomamente il processo di sostituzione graduale dei vecchi Pod con nuovi Pod contenenti il codice aggiornato, seguendo la strategia di aggiornamento configurata (Rolling Update o Recreate, come discusso in precedenza). Questo passaggio chiude il ciclo completo di delivery, portando la modifica dal codice sorgente fino all'ambiente live in produzione, in modo completamente automatico.

Deep Dive Tecnico: Il Player Service (Node.js)

Il backend dedicato alla gestione dell'anagrafica dei giocatori è stato sviluppato utilizzando Node.js, un ambiente di esecuzione JavaScript lato server, insieme al framework Express, una delle librerie più diffuse per la creazione rapida e strutturata di API REST in Node.js.

La scelta di Node.js per questo specifico servizio non è casuale, ma motivata da una caratteristica architetturale precisa del linguaggio: Node.js è single-thread (esegue tutto il codice applicativo su un singolo filo di esecuzione principale), ma è dotato di un Event Loop asincrono e non bloccante. Questo significa che, quando il programma deve eseguire un'operazione potenzialmente lenta — come attendere la risposta di un database remoto — non rimane "bloccato" in attesa, occupando inutilmente risorse: registra invece una funzione di callback da eseguire non appena la risposta arriverà, e nel frattempo torna immediatamente disponibile a gestire altre richieste in arrivo. Questa caratteristica rende Node.js particolarmente efficiente ed eccellente per servizi come il nostro Player Service, che devono gestire un elevato numero di richieste di tipo I/O (Input/Output, principalmente verso il database) senza sprecare memoria o CPU nell'attesa passiva.

Concretamente, quando il Player Service deve interrogare l'istanza PostgreSQL ospitata su Amazon RDS (Relational Database Service, il servizio gestito di AWS per database relazionali), invia la query al database e delega l'attesa della risposta alla rete, senza bloccare l'esecuzione del resto del programma. Nel frattempo, il processo Node.js rimane libero di continuare a servire contemporaneamente altre richieste provenienti da altri utenti.

Un ulteriore aspetto tecnico rilevante riguarda la sicurezza della comunicazione con il database. Trattandosi di un database completamente gestito e ospitato nel cloud (Amazon RDS), è stato necessario configurare correttamente il driver "pg" (il modulo Node.js utilizzato per comunicare con PostgreSQL), implementando l'opzione "ssl: { rejectUnauthorized: false }" all'interno della configurazione di connessione. Questo parametro è un requisito fondamentale imposto dalle configurazioni predefinite di Amazon RDS, ed è necessario per stabilire correttamente una connessione cifrata (SSL/TLS) tra il cluster EKS e l'istanza RDS. La cifratura del traffico in transito è essenziale per prevenire attacchi informatici di tipo Man-in-the-Middle, in cui un malintenzionato posizionato lungo il percorso di rete potrebbe altrimenti intercettare o alterare i dati scambiati tra applicazione e database, incluse potenzialmente le credenziali di accesso.

Deep Dive Tecnico: Il Tournament Service (Python)

Per il motore logico e computazionale del torneo abbiamo selezionato Python come linguaggio di implementazione, affiancato dal micro-framework Flask per l'esposizione delle API REST. Questa scelta è motivata dalle caratteristiche intrinseche del linguaggio: Python è universalmente riconosciuto per la sua eccellenza nella manipolazione di strutture dati complesse — liste, dizionari, insiemi — e per la chiarezza sintattica con cui è possibile esprimere algoritmi matematico-statistici, esattamente il tipo di elaborazione richiesta dal formato di torneo Svizzero.

A differenza del Player Service, che si limita a gestire operazioni relativamente semplici di lettura e scrittura sull'anagrafica, questo servizio agisce come il vero "cervello" computazionale dell'intera applicazione. Espone diversi Endpoint REST (indirizzi API specifici) dedicati, tra cui: uno per avviare la creazione di un nuovo torneo, uno per calcolare e generare gli abbinamenti del round successivo in base ai risultati correnti, e uno per calcolare e restituire la classifica aggiornata comprensiva di tutti i criteri di spareggio.

Poiché il Tournament Service ha la necessità di storicizzare (salvare in modo persistente) lo stato completo e in continua evoluzione delle partite — che, come discusso nella sezione dedicata ai database, non si presta bene alla rigidità di uno schema relazionale predefinito — questo servizio si avvale della libreria ufficiale AWS chiamata "boto3", il Software Development Kit (SDK) Python ufficiale per interagire programmaticamente con tutti i servizi AWS, incluso DynamoDB. Attraverso boto3, il servizio scrive e legge documenti in formato JSON direttamente sulla tabella NoSQL Amazon DynamoDB dedicata ai tornei, sfruttando una caratteristica distintiva di questo database gestito: una latenza di risposta estremamente bassa, tipicamente inferiore ai 10 millisecondi (una "singola cifra di millisecondo"), anche al crescere del numero di tornei e round storicizzati nel tempo — un livello di performance difficile da replicare con un database relazionale tradizionale gestito manualmente.

L'Intelligenza del Sistema: L'Algoritmo Svizzero

La sfida ingegneristica più complessa e interessante affrontata dal Tournament Service è l'implementazione fedele delle vere regole matematiche che governano i giochi competitivi organizzati secondo il sistema Svizzero, un formato ampiamente utilizzato in competizioni ufficiali di giochi come gli scacchi o alcuni giochi di carte collezionabili (TCG) come Magic: The Gathering. A differenza di un torneo casuale, l'algoritmo non genera semplicemente sfide a caso tra i partecipanti, ma segue un insieme di regole precise e verificabili, pensate per garantire equità competitiva anche in un numero limitato di round rispetto al numero totale di partecipanti:

Sorting e Pairing (Ordinamento e Abbinamento)

Prima di ogni nuovo round, tutti i giocatori attivi vengono ordinati in base al punteggio accumulato fino a quel momento (tipicamente 3 punti per vittoria, 1 per pareggio, 0 per sconfitta, secondo le convenzioni più comuni). L'algoritmo procede quindi ad accoppiare tra loro giocatori con punteggio identico o quanto più simile possibile (un principio noto come "Swiss Pairing"), ma con un vincolo aggiuntivo fondamentale: deve verificare, consultando lo storico completo degli incontri già disputati nel torneo in corso, che i due giocatori candidati all'abbinamento non si siano già affrontati in un round precedente. Questo evita ripetizioni ingiuste e garantisce che, con l'avanzare dei round, i giocatori con percorsi simili si scontrino progressivamente tra loro per determinare un classificado finale attendibile.

Gestione del Dispari (BYE)

Poiché il numero totale di giocatori partecipanti a un round potrebbe essere dispari (per esempio a causa di ritiri durante il torneo), non è sempre possibile formare coppie perfette per tutti. In questo caso, l'algoritmo assegna al giocatore rimasto senza avversario — tipicamente quello con il ranking più basso tra i disponibili, e a condizione che non abbia già ricevuto questo trattamento in un round precedente, per garantire equità — un cosiddetto BYE: una vittoria automatica "a tavolino" per quel round, senza dover effettivamente disputare una partita, compensando così l'impossibilità di formare una coppia.

Matematica dei Tie-Breakers (Criteri di Spareggio)

Al termine del torneo, è molto comune che più giocatori terminino con lo stesso punteggio totale, rendendo necessario un metodo oggettivo per stabilire una classifica finale univoca. Per risolvere questi casi di parità, l'algoritmo Python calcola automaticamente due criteri di spareggio standard nel sistema Svizzero:

- **OMW%** (Opponent Match Win Percentage): la percentuale media di vittorie ottenute dagli avversari che un determinato giocatore ha affrontato durante il torneo. Questo criterio premia implicitamente chi ha ottenuto il proprio punteggio affrontando avversari oggettivamente più forti (con un tasso di vittoria più alto), rispetto a chi avesse eventualmente accumulato lo stesso punteggio affrontando avversari più deboli.
- **O-OMW%** (Opponents' Opponent Match Win Percentage): un criterio di secondo livello, calcolato come la media dell'OMW% dei propri avversari, utilizzato per risolvere eventuali ulteriori parità che permangono anche dopo aver applicato il primo criterio.

Il Frontend Web e le Chiamate Asincrone

L'interfaccia utente del nostro progetto è stata progettata seguendo il concetto architetturale di Single Page Application (SPA) "destrutturata", cioè implementata senza l'ausilio di framework complessi come React o Vue, ma direttamente con tecnologie web fondamentali. A differenza di molte applicazioni web tradizionali, non disponiamo di un server applicativo pesante (come potrebbe essere un'applicazione scritta in PHP o Java) che genera dinamicamente l'HTML lato server a ogni richiesta della pagina. Il nostro Frontend è, semplicemente, codice statico — un file HTML, arricchito con JavaScript e CSS — che viene scaricato una sola volta dal browser dell'utente e successivamente renderizzato ed eseguito interamente sul dispositivo dell'utente stesso.

Tutta l'intelligenza dinamica dell'applicazione, cioè la capacità di recuperare e visualizzare dati aggiornati (l'elenco dei giocatori, gli abbinamenti del round corrente, la classifica), è affidata a JavaScript, che utilizza la funzione asincrona nativa del browser chiamata `fetch()` per effettuare chiamate di rete verso i due backend (Player Service e Tournament Service), ottenendo risposte in formato JSON che vengono poi utilizzate per aggiornare dinamicamente il contenuto visualizzato nella pagina, senza mai dover ricaricare l'intera pagina web.

In questa fase architetturale emerge chiaramente l'importanza di due componenti discusse nelle pagine precedenti: il Load Balancer (l'AWS ELB, il punto di ingresso pubblico verso i nostri servizi) e le direttive CORS (Cross-Origin Resource Sharing). Il meccanismo CORS è un'importante misura di sicurezza integrata nei browser moderni, che per impostazione predefinita impedisce a una pagina web caricata da un certo indirizzo (origine) di effettuare liberamente richieste verso un indirizzo di rete diverso (un'altra origine), a meno che il server di destinazione non lo autorizzi esplicitamente. Nel nostro caso, poiché il Frontend, il Player Service e il Tournament Service sono ospitati su indirizzi (o porte) diversi tra loro, è stato necessario configurare esplicitamente Node.js e Python affinché autorizzino le richieste provenienti dall'origine del Frontend, prevenendo così potenziali falle di sicurezza che altrimenti permetterebbero a siti web arbitrari e non autorizzati di effettuare richieste indesiderate verso i nostri backend.